Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Homework 3
SF2524

Aleksi Löf & Anton Blomgren
KTH Royal Institute of Technology

December 9, 2025

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Matrix Computations for Large-scale Systems
SF2524

Contents

AI Usage Specification **1**

Problem 1 **2**

Problem 2 **3**

Problem 3 **5**

Problem 4 **6**

Problem 5 **8**

Problem 6 **9**
a) . . . 9
b) . . . 9
c) . . . 9
d) . . . 9
e) . . . 9
f) . . . 9
g) . . . 10

Problem 7 **11**

References **13**

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Matrix Computations for Large-scale Systems
SF2524

AI Usage Specification

Level 2:
In this report, AI was used to check for spelling and grammar errors in text. It was also used to localize and find errors in code. It was **NOT** used to generate code or text.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 1

According to section *3.1* in *Block 3* but with reversed sorting-order for the eigenvalues, we have that since the convergence rate is approximated by

$$|\beta| = \left|\frac{\lambda_i}{\lambda_j}\right|, \quad i < j, \tag{1}$$

the slowest convergence rate is given by the largest ratio according to (1) that is less than 1. Thus, if λ_m corresponds to the eigenvalue of A with largest magnitude, then, when α is large we have that the error is dominated by the slowest convergence rate given by the ratio $\frac{\lambda_{m-1}}{\lambda_m} \approx 0.99$ in this scenario with $\lambda_m \approx 512$ and $\lambda_{m-1} \approx 507$. With this ratio, the predicted number of iterations can be seen in Figure 1 for the tolerance 10^{-10}.

We can also see that this curve for the predicted number of iterations follows the red curve of the measured number of iteration although slightly below this red curve since we have approximated the convergence rate using the eigenvalues of A. Since the predicted number of iterations is computed with $\ln|\beta|$ in the denominator, when we increase α in the beginning, the ratio β moves closer to value 1 rapidly meaning we will get a larger increase in number of iterations at this stage, as can also be seen for the black dash-dotted curve for smaller α.

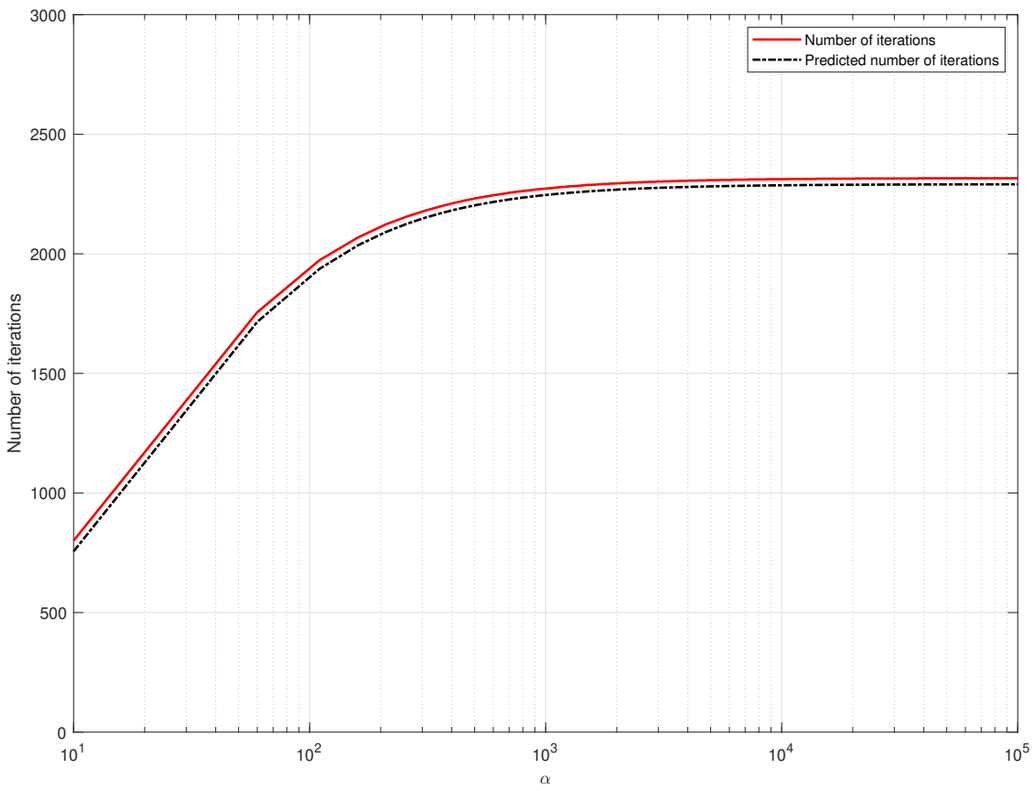


Figure 1: Semilogx plot of iterations as a function of α until error of 10^{-10}.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 2

We want to generalize *Lemma 3.2.3* in *Block 3* and do this by constructing a *Housholder reflector* P where Px is now a multiple of a vector $y \in \mathbb{R}^n \backslash \{0\}$ and for $x \in \mathbb{R}^n \backslash \{0\}$ instead of a multiple of the first unit vector e_1.

Thus, we want

$$Px = \alpha y. \tag{2}$$

From *Definition 2.2.2* in *Block 3*, we have that a *Householder reflector* P is an orthogonal matrix. We can thus find the norm of α in (2) through the preservation of norms for orthogonal matrices in the following way

$$||Px|| = ||x|| = ||\alpha y|| \quad \Rightarrow \quad \alpha = \frac{||x||}{||y||}, \tag{3}$$

which holds since $y \in \mathbb{R}^n \backslash \{0\}$. If we let $\rho = \pm 1$, we can then from (3) define

$$\alpha := \rho \frac{||x||}{||y||}. \tag{4}$$

We now want to show that $P = I - 2uu^T$ according to *Block 3*, *Section 3.2.1*, *Equation (3.3)*. Thus, we first find a normal vector u of a plane such that the the projection of x is in the direction of y, given as follows

$$u = \frac{x - \alpha y}{||x - \alpha y||} \tag{5}$$

which is defined if $x \neq \alpha y$, but where we for this case can avoid the degenerate case by letting the *Householder reflector*, when $x = \alpha y$, be $P_{x=\alpha y} = I$. We then have from (4) and (5)

$$z := x - \alpha y = \begin{bmatrix} x_1 - \rho \frac{||x||}{||y||} y_1 \\ x_2 - \rho \frac{||x||}{||y||} y_2 \\ \vdots \\ x_m - \rho \frac{||x||}{||y||} y_m \end{bmatrix} \quad \Rightarrow \quad u = \frac{z}{||z||}. \tag{6}$$

Letting $P = I - 2uu^T$ and putting this into (2), we get with u in (6)

$$Px = x - 2u(u^T x) = x - 2\frac{z}{||z||}\left(\frac{z^T}{||z||}x\right) = x - 2\frac{z(z^T x)}{z^T z}. \tag{7}$$

From (3) we have $||\alpha y|| = ||x||$ and together with z in (6) and $x^T y = y^T x$, we get

$$z^T z = (x - \alpha y)^T (x - \alpha y) = x^T x - \alpha x^T y - \alpha y^T x + (\alpha y)^T \alpha y = 2(x^T x - \alpha y^T x) \tag{8}$$

also giving

$$z^T x = (x - \alpha y)^T x = x^T x - \alpha y^T x = \frac{1}{2} z^T z. \tag{9}$$

Inserting the results from (8) and (9) into (7) we finally get

$$Px = x - 2\frac{z\left(\frac{1}{2}z^T z\right)}{z^T z} = x - z = x - (x - \alpha y) = \alpha y \tag{10}$$

and therefore, $P \in \mathbb{R}^{n \times n}$ is an orthogonal matrix on the form $P = I - 2uu^T$, satisfying $Px = \alpha y$ with $\alpha := \rho \frac{||x||}{||y||}$. Analogously to *Lemma 3.2.3* in *Block 3*, one can also choose ρ for numerical stability and for P to also be a *Householder reflector* for complex arithmetic.

	CPU-time Algorithm 2	CPU-time of algorithm in (b)
m=10	0.0085 s	0.0114 s
m=100	0.0095 s	0.0241 s
m=200	0.0347 s	0.0923 s
m=300	0.1014 s	0.3010 s
m=400	0.2680 s	1.4922 s

Table 1: Comparison of CPU times for Hessenberg reduction algorithms

In table 1 and in the below figure 2 one can observe that the CPU-time for Algorithm 2 is faster, and remains faster as the m-value grows. The scaling of the Naive Hessenberg method is worse, and one can assume that for large matrices Algorithm 2 is superior in regard to CPU-time.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

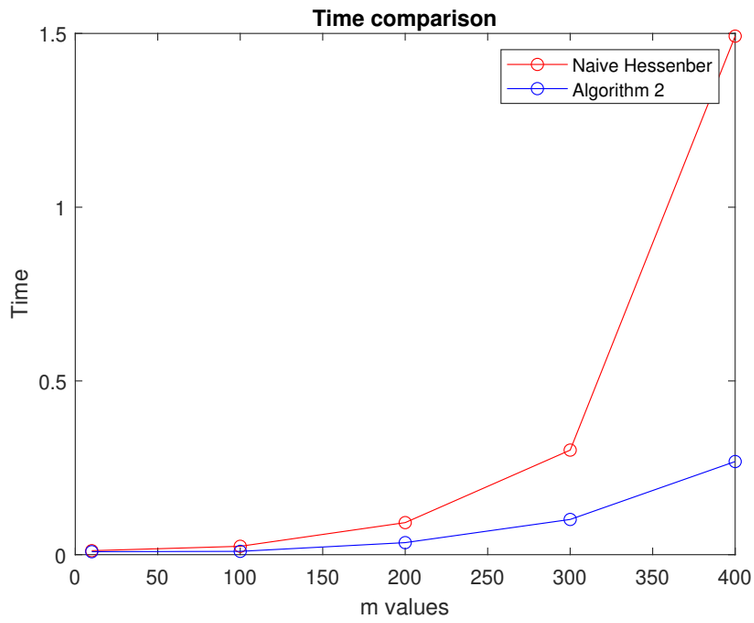


Figure 2: Time comparison for the two methods

ε	$\lvert h_{2,1}\rvert$	
	$\sigma = 0$	$\sigma = a_{2,2}$
0.4	0.0961	0.0769
0.1	0.0311	0.00499
0.01	0.00331	5.00×10^{-5}
0.001	0.000333	5.00×10^{-7}
10^{-4}	3.33×10^{-5}	5.00×10^{-9}
10^{-5}	3.33×10^{-6}	5.00×10^{-11}
10^{-6}	3.33×10^{-7}	5.00×10^{-13}
10^{-7}	3.33×10^{-8}	5.00×10^{-15}
10^{-8}	3.33×10^{-9}	5.00×10^{-17}
10^{-9}	3.33×10^{-10}	5.00×10^{-19}
10^{-10}	3.33×10^{-11}	5.00×10^{-21}
0	0	0

Table 2: Shifted QR-method for two different choices of σ.

We can in Table 2 see a comparison in convergence for the shifted QR-method when we respectively have no shift and a shift corresponding to the last diagonal element of the matrix

$$A = \begin{bmatrix} 3 & 2 \\ \epsilon & 1 \end{bmatrix}, \tag{11}$$

which in this case is $\sigma = a_{2,2} = 1$. Initially one can observe that the convergence rate without a shift $\sigma = 0$ results in a convergence rate of $|\frac{\lambda_2}{\lambda_1}| \approx \frac{1}{3}$ both in the table data and since the eigenvalues can be calculated as

$$\mathbf{det}(A - \lambda I) = 0 \implies \lambda^2 - 4\lambda + 3 - 2\epsilon = 0 \tag{12}$$

This in turn gives the two solutions

$$\lambda_1 = 2 + \sqrt{1 + 2\epsilon} \quad \& \quad \lambda_2 = 2 - \sqrt{1 + 2\epsilon} \tag{13}$$

If $\epsilon \to 0$ the eigenvalues are $\lambda_1 \approx 3$ and $\lambda_2 \approx 1$.

By *Section 3.3* in *Block 3*, the shift QR-method if the shift σ is equal to an eigenvalue λ becomes

$$\overline{H} = RQ + \lambda I = Q^*(H - \lambda I)Q + \lambda I = Q^*HQ \tag{14}$$

In our case, since $\sigma = 1 \approx \lambda_2$, the method behaves similarly to this ideal case, achieving almost quadratic convergence.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 3

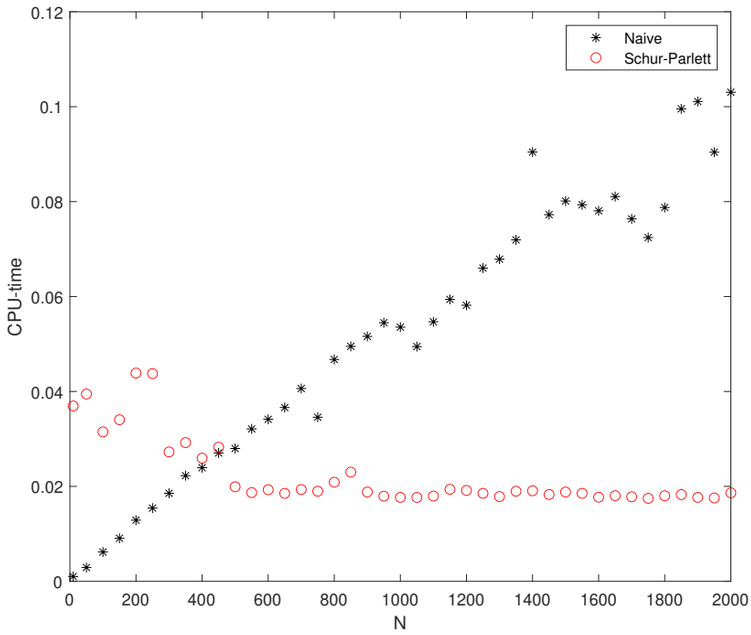


Figure 3: Comparison of time complexity

The naive method computes A^N by performing $N-1$ matrix multiplications and each multiplication of two $n \times n$ matrices costs $O(n^3)$ flops. Thus the total complexity is $O(n^3 N)$ or $p = 3$ and $q = 1$. According to the introduction in *Block 3*, the total complexity of Schur-Parlett method is $O(n^3)$, independent of N, giving $p = 3$ and $q = 0$.

This aligns with what can be observed in figure 3.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 4

Let A be a diagonalizable matrix with eigenvalues λ_i and X an invertible matrix.

$$A = XBX^{-1} \tag{15}$$

where $B = \mathbf{diag}(J_1, ..., J_n)$ is the Jordan canonical form. Since the matrix is diagonalizable, these J_i simplify to 1×1 blocks, which are the eigenvalues of A. Thus it can be written as

$$A = X \cdot \mathbf{diag}(\lambda_1, ..., \lambda_n) \cdot X^{-1} \tag{16}$$

From *Section 4.1.2* in *Block 4*, the matrix function applied in this way also becomes

$$f(A) = f(XBX^{-1}) = Xf(B)X^{-1} = X \cdot \mathbf{diag}(f(\lambda_1), ..., f(\lambda_n)) \cdot X^{-1} \tag{17}$$

For two arbitrary functions p and g it can be written that

$$\begin{aligned} p(A) &= p(XBX^{-1}) = Xp(B)X^{-1} = X \cdot \mathbf{diag}(p(\lambda_1), ..., p(\lambda_n)) \cdot X^{-1} \\ g(A) &= g(XBX^{-1}) = Xg(B)X^{-1} = X \cdot \mathbf{diag}(g(\lambda_1), ..., g(\lambda_n)) \cdot X^{-1} \end{aligned} \tag{18}$$

By problem description of interpolation in eigenvalues $p(\lambda_i) = g(\lambda_i)$, each diagonal element is the same

$$\mathbf{diag}(p(\lambda_1), ..., p(\lambda_n)) = \mathbf{diag}(g(\lambda_1), ..., g(\lambda_n)) \tag{19}$$

Thus we can substitute (10) into (9) and get

$$p(A) = X \cdot \mathbf{diag}(p(\lambda_1), ..., p(\lambda_n)) \cdot X^{-1} = X \cdot \mathbf{diag}(g(\lambda_1), ..., g(\lambda_n)) \cdot X^{-1} = g(A) \tag{20}$$

Therefore, for any diagonalizable matrix A, if a polynomial p interpolates g in the eigenvalues, $p(A) = g(A)$. □

Now, let A be an upper triagular matrix with $\epsilon > 0$ as such

$$A = \begin{bmatrix} \pi & 1 \\ 0 & \pi + \epsilon \end{bmatrix} \tag{21}$$

The eigenvalues for this matrix are $\lambda_1 = \pi$ and $\lambda_2 = \pi + \epsilon$ and the polynomial is $p(z) = \alpha + \beta z$.

For λ_1 we get:

$$\begin{aligned} p(\pi) &= g(\pi) \\ g(\pi) &= \alpha + \beta\pi \end{aligned} \tag{22}$$

For λ_2 we get:

$$\begin{aligned} p(\pi + \epsilon) &= g(\pi + \epsilon) \\ g(\pi + \epsilon) &= \alpha + \beta(\pi + \epsilon) \end{aligned} \tag{23}$$

Subtracting equation (13) from (14) and using some algebra we can quite easily get

$$\begin{aligned} \beta &= \frac{g(\pi + \epsilon) - g(\pi)}{\epsilon} \\ \alpha &= \frac{(\pi + \epsilon)g(\pi) - \pi g(\pi + \epsilon)}{\epsilon} \end{aligned} \tag{24}$$

Using the result that $p(A) = g(A)$ our polynomial $p(z) = \alpha + \beta z$ with *formula 4.2* in *Block 4* and the fact that $g(A) = \exp A$ here

$$\exp A = g(A) = \alpha \cdot I + \beta \cdot A \tag{25}$$

With equations (15) inserted into (16) we get

$$g(A) = \frac{(\pi + \epsilon)g(\pi) - \pi g(\pi + \epsilon)}{\epsilon} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \frac{g(\pi + \epsilon) - g(\pi)}{\epsilon} \begin{bmatrix} \pi & 1 \\ 0 & \pi + \epsilon \end{bmatrix} \tag{26}$$

$$\exp A = \frac{(\pi + \epsilon)e^{\pi} - \pi e^{\pi + \epsilon}}{\epsilon} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \frac{e^{\pi + \epsilon} - e^{\pi}}{\epsilon} \begin{bmatrix} \pi & 1 \\ 0 & \pi + \epsilon \end{bmatrix} \tag{27}$$

The following calculations are simple in character but tedious and are excluded from the report for brevity. In principle we just simplify to get one final matrix, where we did it one element at the time. The result is

$$\exp A = \begin{bmatrix} e^{\pi} & \frac{e^{\pi}(e^{\epsilon} - 1)}{\epsilon} \\ 0 & e^{\pi + \epsilon} \end{bmatrix} \tag{28}$$

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se



Figure 4: Error for different values of ϵ

In Figure 4 we observe that the error becomes smaller when ϵ becomes larger. This can be explained by first looking at the limit

$$\lim_{\epsilon \to 0}(A) = \lim_{\epsilon \to 0} \begin{bmatrix} \pi & 1 \\ 0 & \pi + \epsilon \end{bmatrix} = \begin{bmatrix} \pi & 1 \\ 0 & \pi \end{bmatrix} \tag{29}$$

Here one can see that as $\epsilon \to 0$, A approaches a non-diagonalizable Jordan block, since we only have one linearly independent eigenvector. Also for the application of `FJ=V*F*inv(V)` from *last example in Section 4.1.2* we get

$$FJ = VFV^{-1} = \begin{bmatrix} 1 & 1 \\ 0 & \epsilon \end{bmatrix} \begin{bmatrix} \pi & 0 \\ 0 & \pi + \epsilon \end{bmatrix} \begin{bmatrix} 1 & -\frac{1}{\epsilon} \\ 0 & \frac{1}{\epsilon} \end{bmatrix} \tag{30}$$

Here we observe that if $\epsilon \to 0$ the values of X grow larger and larger, causing the conditioning of X to become worse.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 5

Table 3: Number of matrix-matrix products

m\j	0	1	2	3	4	5	6	7
1	0	1	2	3	4	5	6	7
2	1	2	3	4	5	6	7	8
3	2	3	4	5	6	7	8	9
4	3	4	5	6	7	8	9	10
5	4	5	6	7	8	9	10	11
6	5	6	7	8	9	10	11	12
7	6	7	8	9	10	11	12	13

Table 4: $\log_{10}(\|P_{j,m} - \exp(A)\|)$

m\j	0	1	2	3	4	5	6	7
1	0.4424	-0.1193	-1.1479	-1.4552	-1.7618	-2.0659	-2.3686	-2.6704
2	0.6262	-0.1375	-1.4032	-2.2803	-2.9631	-3.5991	-4.2171	-4.8269
3	0.6495	-1.0885	-2.2179	-3.2517	-4.2318	-5.1741	-6.0968	-7.0097
4	0.5573	-1.2054	-2.8916	-4.2727	-5.5615	-6.8077	-8.0327	-9.2473
5	0.3743	-1.8960	-3.7009	-5.3789	-6.9706	-8.5189	-10.0456	-11.5615
6	0.1165	-2.3960	-4.5710	-6.5541	-8.4483	-10.2984	-12.1265	-13.9313
7	-0.2050	-3.0399	-5.5034	-7.7894	-9.9857	-12.1374	-14.2542	-14.1888

Table 5: CPU time (seconds)

m\j	0	1	2	3	4	5	6	7
1	0.0009	0.0006	0.0009	0.0011	0.0010	0.0009	0.0011	0.0018
2	0.0007	0.0010	0.0010	0.0012	0.0015	0.0015	0.0017	0.0020
3	0.0008	0.0011	0.0014	0.0016	0.0019	0.0021	0.0020	0.0023
4	0.0011	0.0013	0.0015	0.0017	0.0019	0.0021	0.0023	0.0025
5	0.0015	0.0015	0.0021	0.0019	0.0022	0.0024	0.0025	0.0028
6	0.0016	0.0017	0.0020	0.0021	0.0023	0.0026	0.0028	0.0030
7	0.0017	0.0018	0.0021	0.0023	0.0025	0.0026	0.0029	0.0033

If an error smaller than 10^{-12} is desired, the number of matrix-matrix products needs to be atleast 11 according to Table 3 and Table 4. This corresponds to 0.0026 seconds or more according to Table 5. For Taylor approximation, when $j = 0$, we get $\log_{10}(||P_{0,25} - \exp(A)||) = -12.067$ for the CPU-time around 0.0070 seconds, meaning for Taylor approximation we need 24 matrix-matrix multiplications to achieve an error smaller than 10^{-12}.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 6

a) 312: Householder reflectors that map x to the direction of e1

Given some vector $x \in \mathbb{R}^n \setminus \{0\}$, we want to find a P matrix that satisfies

$$Px = \alpha e_1 \tag{31}$$

In words, we want to find a P which reflects x onto the first unit vector. Here the "Household reflector" P is defined as $P = I - 2uu^T$. In the video it is subsequently shown that this is true, by inserting the definition of P into the LHS of 31. Then it is shown that $\frac{z^T x}{z^T z} = 1$, where $z = x - \alpha e_1$ and this proves Lemma to be true.

b) QR-factorization of Hessenberg via Givens derivation

We want to perform a QR-factorization of a Hessenberg matrix A and we do this through the application of *Givens Rotators* on phase 2 of the QR-method. This means that we will factorize A into a multiplication of an orthogoanl matrix, consisting of a *Givens Rotator* product $G_1 \dots G_{n-1}$, and an upper triangular matrix H_n. For this to be possible, it is necessary to assume that the Hessenberg matrix A is *unreduced*, meaning that all the elements on the first sub-diagonal are non-zero. This unreduced behavior for A is given by iteratively creating an upper triangular structure for H_n by the recursion $H_{i+1} = G_i^T H_i$, $i = 1, ..., n-1$ where G_i is the givens rotator for the previous block in which we want to select c_i and s_i such that we get the desired upper triangular structure. By letting $A = H_1$, which already has upper triangular structure, we can then use the orthogonality of the *Givens Rotators* to get the desired QR-factorization by multiplying the product $G_1 \dots G_{n-1}$ on both sides of $H_n = G_{n-1}^T \dots G_1 H_1$.

c) 333: Shifted QR-method with exact shifts

How do we select the shift μ for the shifted QR-method? In the video it is called an "exact" shift since we want to select $\mu =$ eigenvalue of A $= \lambda_i$. The eigenvalue is want we are computing though, so we don't have access a priori. For the shifted QR-method we will have convergence in one step if $\mu = \lambda_i$, we can't do this in practise though. But, we hope to use an eigenvalue approximation and get fast convergence.

d) 344 Convergence of QR-method. Part 1. Simultaneous itera- tion: Normalized unnormalized

The *Simultaneous Iteration* (*SI*) is a generalized version of the *Power Method* where we now want to handle matrices, simultaneously handling several vectors, instead of just one vector at a time. The method comes in an unnormalized and normalized version. It is shown that for the *PM*, the unnormalized version has the same iterates as the normalized version.

Similarly to the unnormalized *PM* iterates, by instead letting the starting vector be a starting matrix, one can easily obtain the unnormalized *SI* method iterates in the same way.

For the unnormalized *SI* iterates, one needs that the starting matrix is orthogonal and we need to generalize the normalization process through a QR-factorization of the unnormalized iteration. It is then shown through this QR-factorization that the unnormalized *SI* and normalized *SI* have the same iterates if the upper triangular matrix $R = R^{(n)} \dots R^{(1)}$ is invertible, meaning all the iterates must have full rank, and if the diagonal elements of R are all positive.

e) 345 Convergence of QR-method. Part 2. Convergence unnor- malized simultaneous iteration for special case (Week 4 quiz 6)

By looking at the seperate columns of the QR-factorization of $A = QR$ together with convergence theory of the Power Method, the video shows that the columns q_i approach the eigenvectors of A. They do so with some error factor, for example for q_2 we get

$$q_2^k = e_2 + \mathcal{O}(\frac{\lambda_2}{\lambda_1})^k + \mathcal{O}(\frac{\lambda_3}{\lambda_2})^k \tag{32}$$

Where e_2 is the second eigenvector of A and this pattern continous for q_i, thus q_i converge to the eigenvalues with some error terms.

f) 423 Newton SQRTM

One specialized method for computing the principal square root of a matrix A is through the *Newton method*. To do this we have to first assume that the eigenvalues of the matrix A does not lie on the negative part of the real axis.

A scalar version of the *Newton method* is first considered giving a formula for the Newton step and is then translated to a matrix version by replacing the scalar elements with matrices.

It can then be proven for the convergence of the matrix version of the *Newton method* that all the iterates can be diagonalized simultaneously if one assumes that the matrix A we want to compute the square root of is symmetric (although works for other cases than symmetric as well). We thus have $A = V \Lambda V^{-1}$ for an upper triangular matrix V which diagonalizes A and which is also proven to diagonalize all iterates. We therefore have an update formula for the diagonal matrix giving iterates with diagonal elements corresponding to updates from the Newton method applied to the scalar problem. This results in inhereted convergence from the scalar version and since the Λ matrices converge and diagonalize the square root iterates, these also converge leading to local quadratic convergence at the least.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

g) 302: The two-phase approach

The standard QR-method can be slow in two ways, it needs many iterations to rach upper triangular form, and doing one step is very expensive, $\mathcal{O}(n^3)$. Thus we want to improve with the "Two phase approach". This entails performing two tranformations of the matrix A, in the first step $A = QHQ^T$ and in the second PTP^T. Thus with this tranformation we tranform A to T like

$$A = QPT(QP)^T \tag{33}$$

In phase 1 we use household reflectors and in phase 2 we perform QR-method on a Hessenberg matrix.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

Problem 7

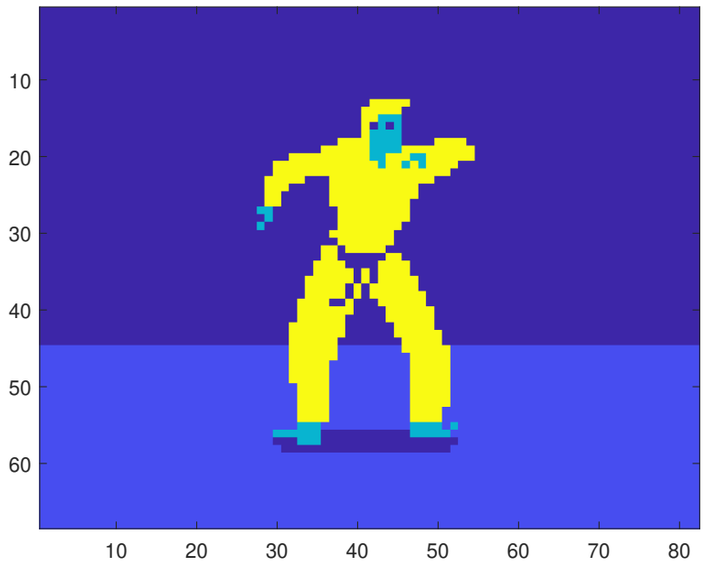


Figure 5: Favourite *Ninja Works It* image (image 12)

When we loop through the images we get a video of an 8-bit Ninja dancing.

After some detective work we can conclude that the data set is called *Ninja Works It* and the music that should be playing in the background is an 8-bit cover of *Daft Punk's* song *Harder, Better, Faster, Stronger* and that the dance is from *M.C. Hammer's* music video to their song *U can't Touch This*. The 8-bit Ninja dancing meme is originally based on an animation from the 1984 Atari game *Karateka*, and originally posted 23rd October 2004 on the meme-community website YTMND. All these facts are according to [1] and [2]

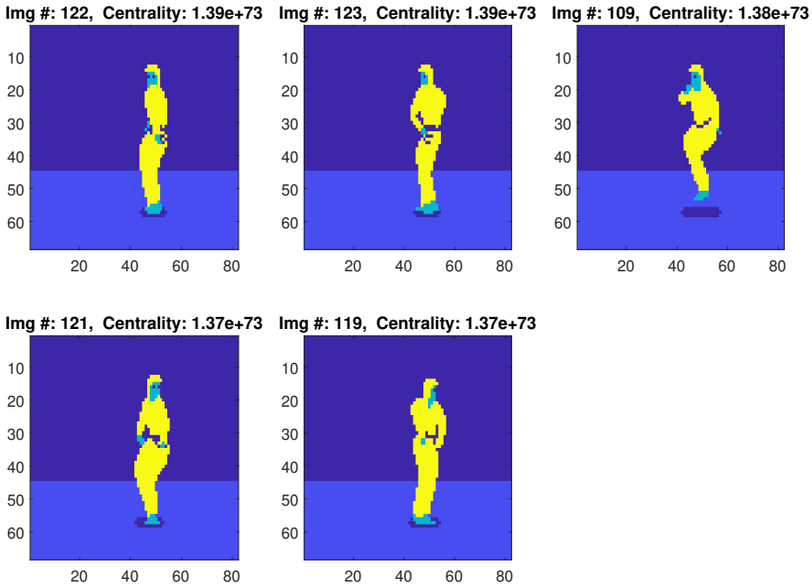


Figure 6: 5 most central images.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

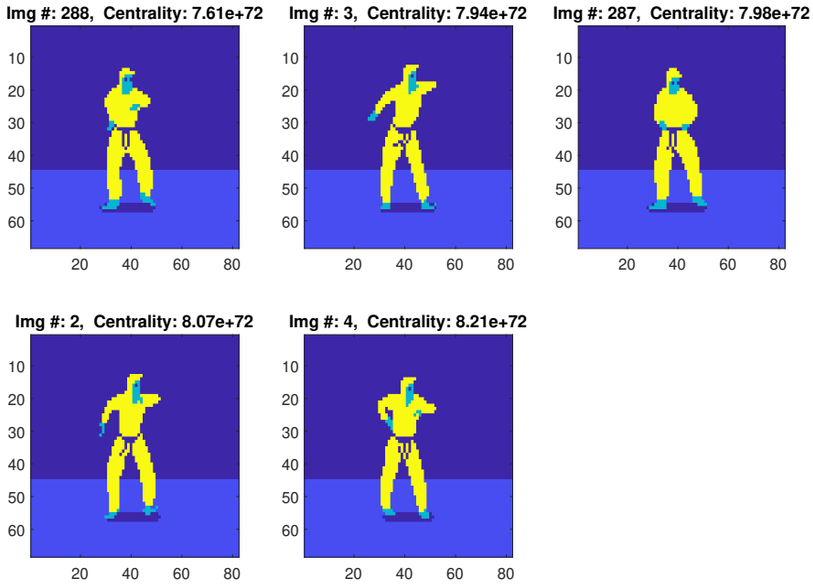


Figure 7: 5 least central images.

Avg Time `expm`	6.424×10^{-3}
Avg Time truncated Taylor	1.329×10^{-3}
$\log_{10}(\|P_{j,m} - \exp(A)\|)$	7.553×10^{1}
First image where rankings differ	position 27
`expm` method ranks image	115 at position 27
Taylor method ranks image	185 at position 27

Table 6: Comparison of MATLAB expm and Taylor method

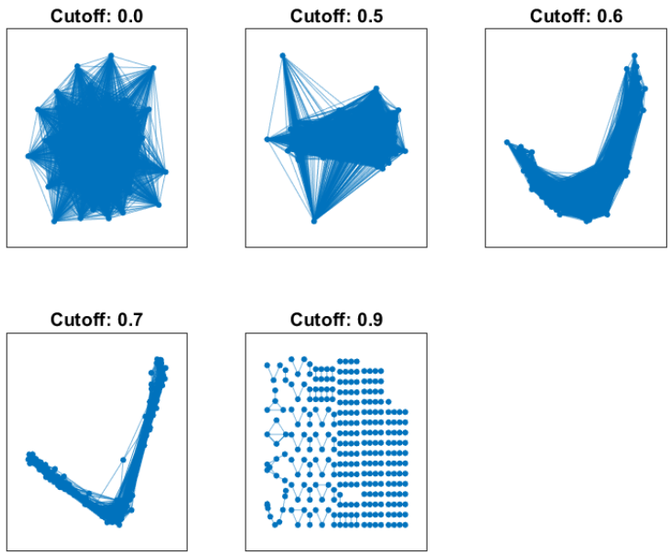


Figure 8: Graph plot for different cutoff values.

Aleksi Löf
Anton Blomgren
aleksil@kth.se
ablomg@kth.se

References

[1] *Ninja Works It | Know Your Meme — knowyourmeme.com.* `https://knowyourmeme.com/memes/ninja-works-it`. [Accessed 02-12-2025].

[2] *Ninja works it - YTMND — wiki.ytmnd.com.* `https://wiki.ytmnd.com/Ninja_works_it`. [Accessed 02-12-2025].